```
!pip install xlrd openpyxl
```

```
Requirement already satisfied: xlrd in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: openpyxl in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: et-xmlfile in /usr/local/lib/python3.12/dist-p
```

```
import pandas as pd
import numpy as np
import sqlite3

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, class
```

```
df = pd.read_csv("UCI_Credit_Card.csv")

print("Shape:", df.shape)
df.head()
```

```
Shape: (30000, 25)
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	.
0	1	20000.0	2	2	1	24	2	2	-1	-1	
1	2	120000.0	2	2	2	26	-1	2	0	0	
2	3	90000.0	2	2	2	34	0	0	0	0	
3	4	50000.0	2	2	1	37	0	0	0	0	
4	5	50000.0	1	2	1	57	-1	0	-1	0	

5 rows × 25 columns

```
# Cleaning column names

df = df.rename(columns={
    "default.payment.next.month": "DEFAULT"
})

# Removing ID because it is not useful for prediction
df = df.drop(columns=["ID"])

print(df.columns)
print(df.isnull().sum())
```

```
Index(['LIMIT_BAL', 'SEX', 'EDUCATION', 'MARRIAGE', 'AGE', 'PAY_0', 'PAY_2',
       'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6', 'BILL_AMT1', 'BILL_AMT2',
       'BILL_AMT3', 'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6', 'PAY_AMT1',
       'PAY_AMT2', 'PAY_AMT3', 'PAY_AMT4', 'PAY_AMT5', 'PAY_AMT6', 'DEFAULT'],
      dtype='object')
LIMIT_BAL    0
SEX          0
EDUCATION    0
MARRIAGE     0
AGE          0
PAY_0        0
PAY_2        0
PAY_3        0
PAY_4        0
PAY_5        0
PAY_6        0
BILL_AMT1    0
BILL_AMT2    0
BILL_AMT3    0
BILL_AMT4    0
BILL_AMT5    0
BILL_AMT6    0
PAY_AMT1     0
PAY_AMT2     0
PAY_AMT3     0
PAY_AMT4     0
PAY_AMT5     0
PAY_AMT6     0
DEFAULT      0
dtype: int64
```

```
# Creating simple risk features

df["TOTAL_BILL"] = df[
    ["BILL_AMT1", "BILL_AMT2", "BILL_AMT3", "BILL_AMT4", "BILL_AMT5
].sum(axis=1)

df["TOTAL_PAYMENT"] = df[
    ["PAY_AMT1", "PAY_AMT2", "PAY_AMT3", "PAY_AMT4", "PAY_AMT5", "P
].sum(axis=1)

df["AVG_PAYMENT_DELAY"] = df[
    ["PAY_0", "PAY_2", "PAY_3", "PAY_4", "PAY_5", "PAY_6"]
].mean(axis=1)

df["CREDIT_USAGE_RATIO"] = df["TOTAL_BILL"] / df["LIMIT_BAL"]

df.head()
```

	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5
0	20000.0	2	2	1	24	2	2	-1	-1	-2
1	120000.0	2	2	2	26	-1	2	0	0	0
2	90000.0	2	2	2	34	0	0	0	0	0
3	50000.0	2	2	1	37	0	0	0	0	0
4	50000.0	1	2	1	57	-1	0	-1	0	0

5 rows × 28 columns

```
# Basic risk summary

total_customers = len(df)
default_customers = df["DEFAULT"].sum()
default_rate = df["DEFAULT"].mean() * 100
avg_credit_limit = df["LIMIT_BAL"].mean()

print("Total Customers:", total_customers)
print("Default Customers:", default_customers)
print("Default Rate:", round(default_rate, 2), "%")
print("Average Credit Limit:", round(avg_credit_limit, 2))
```

```
Total Customers: 30000
Default Customers: 6636
Default Rate: 22.12 %
Average Credit Limit: 167484.32
```

```
# Preparing data for ML

X = df.drop(columns=["DEFAULT"])
y = df["DEFAULT"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Logistic Regression model

log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_scaled, y_train)
```

```
5
log_pred = log_model.predict(X_test_scaled)

print("Logistic Regression Accuracy:", accuracy_score(y_test, log_
print(confusion_matrix(y_test, log_pred))
print(classification_report(y_test, log_pred))
```

```
Logistic Regression Accuracy: 0.8096666666666666
[[4534  139]
 [1003  324]]
              precision    recall  f1-score   support

           0       0.82      0.97      0.89      4673
           1       0.70      0.24      0.36      1327

    accuracy                           0.81      6000
   macro avg       0.76      0.61      0.63      6000
weighted avg       0.79      0.81      0.77      6000
```

```
# Random Forest model

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight="balanced"
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))
print(confusion_matrix(y_test, rf_pred))
print(classification_report(y_test, rf_pred))
```

```
Random Forest Accuracy: 0.8098333333333333
[[4414  259]
 [ 882  445]]
              precision    recall  f1-score   support

           0       0.83      0.94      0.89      4673
           1       0.63      0.34      0.44      1327

    accuracy                           0.81      6000
   macro avg       0.73      0.64      0.66      6000
weighted avg       0.79      0.81      0.79      6000
```

```
# Feature importance

importance = pd.DataFrame({
    "Feature": X.columns,
```

```
    "Importance": rf_model.feature_importances_
}).sort_values(by="Importance", ascending=False)

importance.head(15)
```

	Feature	Importance
5	PAY_0	0.072940
25	AVG_PAYMENT_DELAY	0.063163
26	CREDIT_USAGE_RATIO	0.056864
4	AGE	0.053910
24	TOTAL_PAYMENT	0.053887
0	LIMIT_BAL	0.048557
11	BILL_AMT1	0.047748
23	TOTAL_BILL	0.046681
12	BILL_AMT2	0.041791
18	PAY_AMT2	0.040062
17	PAY_AMT1	0.039860
19	PAY_AMT3	0.039264
14	BILL_AMT4	0.038990
13	BILL_AMT3	0.038698
16	BILL_AMT6	0.038041

Next steps: Generate code with `importance` New interactive sheet

```
# Adding prediction results for Power BI

df["PREDICTED_DEFAULT"] = rf_model.predict(X)

df["RISK_LEVEL"] = np.where(
    df["PREDICTED_DEFAULT"] == 1,
    "High Risk",
    "Low Risk"
)

df.head()
```

	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5
0	20000.0	2	2	1	24	2	2	-1	-1	-2
1	120000.0	2	2	2	26	-1	2	0	0	0
2	90000.0	2	2	2	34	0	0	0	0	0
3	50000.0	2	2	1	37	0	0	0	0	0
4	50000.0	1	2	1	57	-1	0	-1	0	0

5 rows × 30 columns

```
# Export clean file for Power BI

df.to_csv("financial_risk_powerbi.csv", index=False)
importance.to_csv("feature_importance.csv", index=False)

print("Files created:")
print("financial_risk_powerbi.csv")
print("feature_importance.csv")
```

```
Files created:
financial_risk_powerbi.csv
feature_importance.csv
```

```
# using SQLite

conn = sqlite3.connect("financial_risk.db")

df.to_sql("credit_risk", conn, if_exists="replace", index=False)

query = """
SELECT
    RISK_LEVEL,
    COUNT(*) AS total_customers,
    ROUND(AVG(LIMIT_BAL), 2) AS avg_credit_limit,
    ROUND(AVG(CREDIT_USAGE_RATIO), 2) AS avg_credit_usage
FROM credit_risk
GROUP BY RISK_LEVEL;
"""

sql_result = pd.read_sql(query, conn)
sql_result
```

	RISK_LEVEL	total_customers	avg_credit_limit	avg_credit_usage
0	High Risk	6020	125777.41	2.79
1	Low Risk	23980	177954.53	2.10

Next steps: Generate code with `sql_result` New interactive sheet

```
from google.colab import files

files.download("financial_risk_powerbi.csv")
files.download("feature_importance.csv")
files.download("financial_risk.db")
```